

Introduction to Programming with Haskell

Recursive functions

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Recursive definitions

- **Base case:** $f(0)$ is given

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$
- Factorial

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$
- Factorial
 - $0! = 1$

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$
- Factorial
 - $0! = 1$
 - For $n > 0$, $n! = n \times (n-1)!$

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer  
factorial 0 = 1  
factorial n = n * factorial (n-1)
```

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- **Integer** represents integers of arbitrarily large magnitude

```
factorial 40
815915283247897734345611269596115894272000000000
```


Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- **Integer** represents integers of arbitrarily large magnitude

```
factorial 40
815915283247897734345611269596115894272000000000
```

- **Int** has lower and upper bounds

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- **Integer** represents integers of arbitrarily large magnitude

```
factorial 40
8159152832478977343456112695961158942720000000000
```

- **Int** has lower and upper bounds
- -2^{63} and $2^{63} - 1$ on a 64-bit machine

Example program: Factorial

- Can write in different styles!

```
factorial :: Integer → Integer
factorial n = if n == 0 then 1 else n * factorial (n-1)
```

```
factorial n
  | n == 0    = 1
  | otherwise = n * factorial (n-1)
```

```
factorial n = case n of
  | 0 → 1
  | _ → n * factorial (n-1)
```

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?
 - Note the parentheses again!

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?
 - Note the parentheses again!
 - `factorial -1` is `1` subtracted from the function `factorial`

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?
 - Note the parentheses again!
 - `factorial -1` is `1` subtracted from the function `factorial`
 - Type error!

Example program: Factorial

- Fix the factorial function to work on negative values

```
factorial :: Integer → Integer
factorial n
  | n ≤ 0    = 1
  | otherwise = n * factorial (n-1)
factorial (-20)
1
```

Example program: Factorial

- Can print an error message and abort on negative values

```
factorial :: Integer → Integer
factorial n
  | n == 0    = 1
  | n > 0     = n * factorial (n-1)
  | otherwise = error "Negative input"
factorial (-20)
  *** Exception: Negative input
```

Example program: Factorial

- Can print an error message and abort on negative values

```
factorial :: Integer → Integer
factorial n
  | n == 0    = 1
  | n > 0     = n * factorial (n-1)
  | otherwise = error "Negative input"
factorial (-20)
  *** Exception: Negative input
```

- **error** has type **String** → a

Example program: Factorial

- Can print an error message and abort on negative values

```
factorial :: Integer → Integer
factorial n
  | n == 0    = 1
  | n > 0     = n * factorial (n-1)
  | otherwise = error "Negative input"
factorial (-20)
  *** Exception: Negative input
```

- **error** has type **String** → a
- Prints the error message and abort

Example program: Factorial

- Can print an error message and abort on negative values

```
factorial :: Integer → Integer
factorial n
  | n == 0    = 1
  | n > 0     = n * factorial (n-1)
  | otherwise = error "Negative input"
factorial (-20)
  *** Exception: Negative input
```

- **error** has type **String** → a
- Prints the error message and abort
- **Input** – Error message is the input **Output** – None, but the output type matches any type

Example program: gcd

- Euclid's algorithm for computing the greatest common divisor

Example program: gcd

- Euclid's algorithm for computing the greatest common divisor
- Assume a and b are not both zero

Example program: gcd

- Euclid's algorithm for computing the greatest common divisor
- Assume a and b are not both zero
- Handle negative integers appropriately

Example program: gcd

- Euclid's algorithm for computing the greatest common divisor
- Assume *a* and *b* are not both zero
- Handle negative integers appropriately
- Haskell function `myGCD`

```
myGCD :: Integer → Integer → Integer
myGCD a b
  | a < 0 || b < 0    = myGCD (abs a) (abs b)
  | b == 0            = a
  | otherwise        = myGCD b (a `mod` b)
```

Example program: gcd

- Euclid's algorithm for computing the greatest common divisor
- Assume *a* and *b* are not both zero
- Handle negative integers appropriately
- Haskell function `myGCD`

```
myGCD :: Integer → Integer → Integer
myGCD a b
  | a < 0 || b < 0    = myGCD (abs a) (abs b)
  | b == 0            = a
  | otherwise         = myGCD b (a `mod` b)
```

- `abs :: Integer → Integer` returns the absolute value of an integer

Example program: gcd

- Euclid's algorithm for computing the greatest common divisor
- Assume *a* and *b* are not both zero
- Handle negative integers appropriately
- Haskell function `myGCD`

```
myGCD :: Integer → Integer → Integer
myGCD a b
  | a < 0 || b < 0    = myGCD (abs a) (abs b)
  | b == 0            = a
  | otherwise        = myGCD b (a `mod` b)
```

- `abs :: Integer → Integer` returns the absolute value of an integer
- `gcd` is part of the [standard library](#), hence the renaming!

Example: power of five

- Check if the input is a power of five

```
isPOF :: Integer → Bool
isPOF n
  | n ≤ 0      = False
  | n == 1     = True
  | otherwise  = n `mod` 5 == 0 && isPOF (n `div` 5)
```

Example program: smallest divisor

- Find the smallest divisor of n which greater than 1

Example program: smallest divisor

- Find the smallest divisor of n which greater than 1
- **Strategy:** try 2, 3, ...

Example program: smallest divisor

- Find the smallest divisor of n which greater than 1
- **Strategy:** try 2, 3, ...
- In the worst case, stop at n

Example program: smallest divisor

- Find the smallest divisor of n which greater than 1
- **Strategy:** try 2, 3, ...
- In the worst case, stop at n
- Haskell function `smallestDiv`

```
smallestDiv :: Integer → Integer
```

```
smallestDiv n = divSearch n 2
```

```
divSearch :: Integer → Integer → Integer
```

```
divSearch m i
```

```
    | m `mod` i == 0    = i
```

```
    | otherwise        = divSearch m (i+1)
```

Local definitions

- `divSearch` is a helper function

Local definitions

- `divSearch` is a helper function
- Might not be invoked independently

Local definitions

- `divSearch` is a helper function
- Might not be invoked independently
- So we can make the definition **local**

```
smallestDiv :: Integer → Integer
smallestDiv n = divSearch n 2 where
    divSearch :: Integer → Integer → Integer
    divSearch m i
        | m `mod` i == 0    = i
        | otherwise        = divSearch m (i+1)
```

Local definitions

- Local functions can use names defined in the surrounding context

Local definitions

- Local functions can use names defined in the surrounding context
- The first argument of `divSearch`, `m`, never changes

Local definitions

- Local functions can use names defined in the surrounding context
- The first argument of `divSearch`, `m`, never changes
- It is in fact the argument of `smallestDiv`

Local definitions

- Local functions can use names defined in the surrounding context
- The first argument of `divSearch`, `m`, never changes
- It is in fact the argument of `smallestDiv`
- Simplified `divSearch`:

```
smallestDiv :: Integer → Integer
smallestDiv n = divSearch 2 where
    divSearch :: Integer → Integer
    divSearch i
        | n `mod` i == 0    = i
        | otherwise        = divSearch (i+1)
```

Local definitions

- Can also use **let** to define local functions

```
smallestDiv :: Integer → Integer
smallestDiv n =
    let divSearch i
        | n `mod` i == 0    = i
        | otherwise        = divSearch (i+1)
    in divSearch (n-1)
```

Local definitions

- What is the largest divisor of n , other than n itself?

```
largestDiv :: Integer → Integer  
largestDiv n = div n (smallestDiv n)
```

Local definitions

- What is the largest divisor of n , other than n itself?

```
largestDiv :: Integer → Integer  
largestDiv n = div n (smallestDiv n)
```

- Instead of `div n (smallestDiv n)` we can also write `div n $ smallestDiv n`

Local definitions

- What is the largest divisor of n , other than n itself?

```
largestDiv :: Integer → Integer  
largestDiv n = div n (smallestDiv n)
```

- Instead of `div n (smallestDiv n)` we can also write `div n $ smallestDiv n`
- `$` helps reduce clutter involving nested parentheses:

Local definitions

- What is the largest divisor of n , other than n itself?

```
largestDiv :: Integer → Integer  
largestDiv n = div n (smallestDiv n)
```

- Instead of `div n (smallestDiv n)` we can also write `div n $ smallestDiv n`
- `$` helps reduce clutter involving nested parentheses:
- `f $ g $ h $ x+1` instead of `f (g (h (x+1)))`

Example: length of an integer

- The number of digits in a non-negative integer n

Example: length of an integer

- The number of digits in a non-negative integer n
- If $n < 0$, it is an error

Example: length of an integer

- The number of digits in a non-negative integer n
- If $n < 0$, it is an error
- Otherwise, determine the number of digits in $n \text{ div } 10$ and add 1

Example: length of an integer

- The number of digits in a non-negative integer n
- If $n < 0$, it is an error
- Otherwise, determine the number of digits in $n \text{ div } 10$ and add 1
- Haskell function `intLength`

```
intLength :: Integer → Integer
intLength n
  | n < 0      = error "Negative input"
  | n < 10     = 1
  | otherwise  = 1 + intLength (n `div` 10)
```

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)
- `intReverse 13276` should give `67231`

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)
- `intReverse 13276` should give `67231`
- **Strategy:**

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)
- `intReverse 13276` should give `67231`
- **Strategy:**
 - Split `13276` as `1327` and `6` using `div` and `mod`

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)
- `intReverse 13276` should give `67231`
- **Strategy:**
 - Split `13276` as `1327` and `6` using `div` and `mod`
 - Recursively reverse `1327` to get `7231`

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)
- `intReverse 13276` should give `67231`
- **Strategy:**
 - Split `13276` as `1327` and `6` using `div` and `mod`
 - Recursively reverse `1327` to get `7231`
 - Multiply `6` by a suitable power of `10` and add: $60000 + 7231 = 67231$

Example: reverse a number

- `intReverse n` reverses the digits of `n` (non-negative)
- `intReverse 13276` should give `67231`
- **Strategy:**
 - Split `13276` as `1327` and `6` using `div` and `mod`
 - Recursively reverse `1327` to get `7231`
 - Multiply `6` by a suitable power of `10` and add: $60000 + 7231 = 67231$
 - Use `intLength` to determine the power of `10`

Example: reverse a number

```
intReverse :: Integer → Integer
intReverse n
  | n < 0      = error "Negative input"
  | n < 10     = n
  | otherwise = intReverse (n `div` 10) +
                  (n `mod` 10) *
                  power 10 (intLength n - 1)

power :: Integer → Integer → Integer
power m 0 = 1
power m n = m * power m (n-1)
```

Example: a better `intReverse`

- Reverse `n` **into** an accumulator `a`

Example: a better `intReverse`

- Reverse `n` **into** an accumulator `a`
- Repeatedly extract the last digit of `n` and add at the end of `a`

Example: a better `intReverse`

- Reverse `n` **into** an accumulator `a`
- Repeatedly extract the last digit of `n` and add at the end of `a`
- This automatically reverses `n` at the end of `a`

Example: a better `intReverse`

- Reverse `n` **into** an accumulator `a`
- Repeatedly extract the last digit of `n` and add at the end of `a`
- This automatically reverses `n` at the end of `a`
- Start with `a` set to `0`

Example: a better `intReverse`

- Reverse `n` **into** an accumulator `a`
- Repeatedly extract the last digit of `n` and add at the end of `a`
- This automatically reverses `n` at the end of `a`
- Start with `a` set to `0`
- Use `divMod` to extract last digit

Example: a better `intReverse`

- Reverse `n` **into** an accumulator `a`
- Repeatedly extract the last digit of `n` and add at the end of `a`
- This automatically reverses `n` at the end of `a`
- Start with `a` set to `0`
- Use `divMod` to extract last digit
- `divMod n p = (div n p, mod n p)`

Example: a better `intReverse`

```
intReverse :: Integer → Integer
intReverse n = if n < 0
  then error "Negative input"
  else revInto 0 n

revInto :: Integer → Integer → Integer
revInto a n =
  let (d,m) = n `divMod` 10
  in if n == 0 then a else revInto (10*a+m) d
```